

# Digital System Design

## LAB III Modelling Styles

**DONG Yunyang**

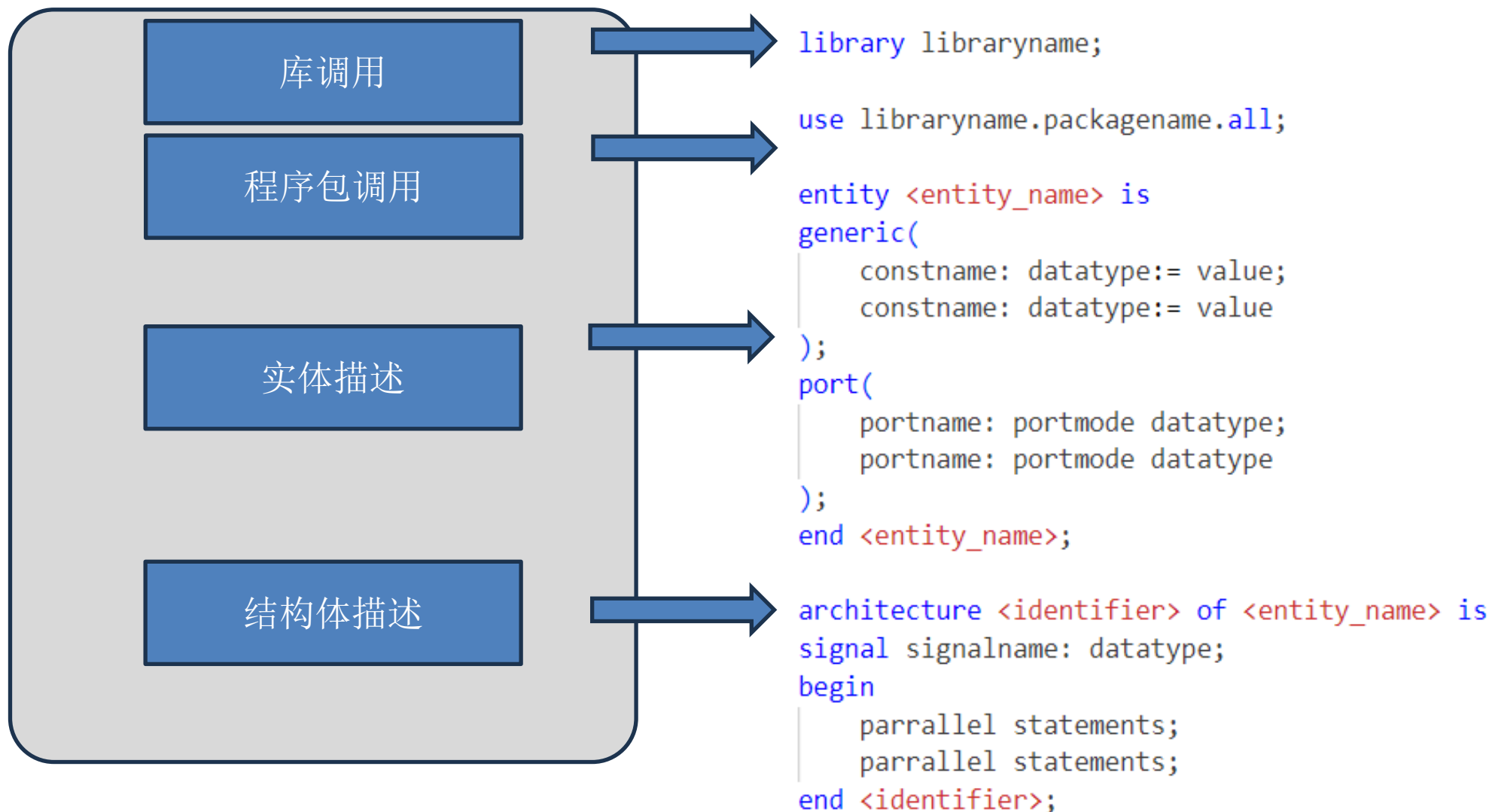
**[dongyy@sustech.edu.cn](mailto:dongyy@sustech.edu.cn)**

**411, No. 2, Hui Yuan**

**Tencent Meeting: 874-068-9694**

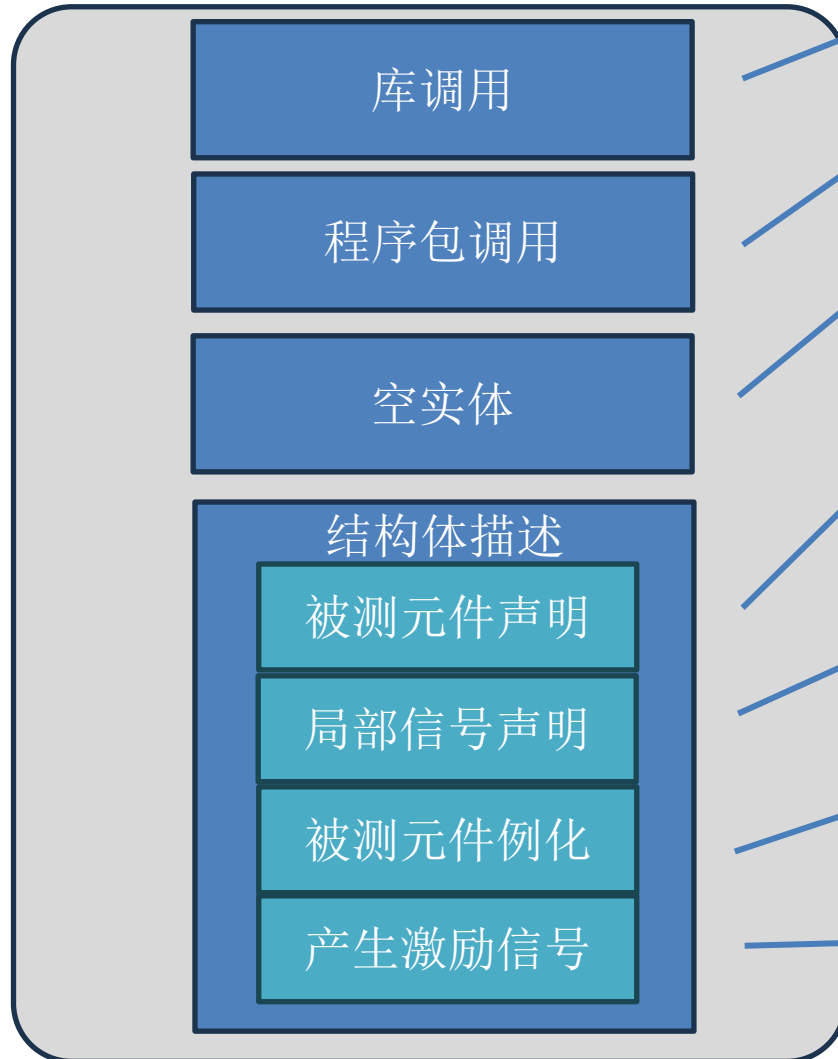
# Review - VHDL

## VHDL基本结构



# Review - Testbench

Testbench基本结构



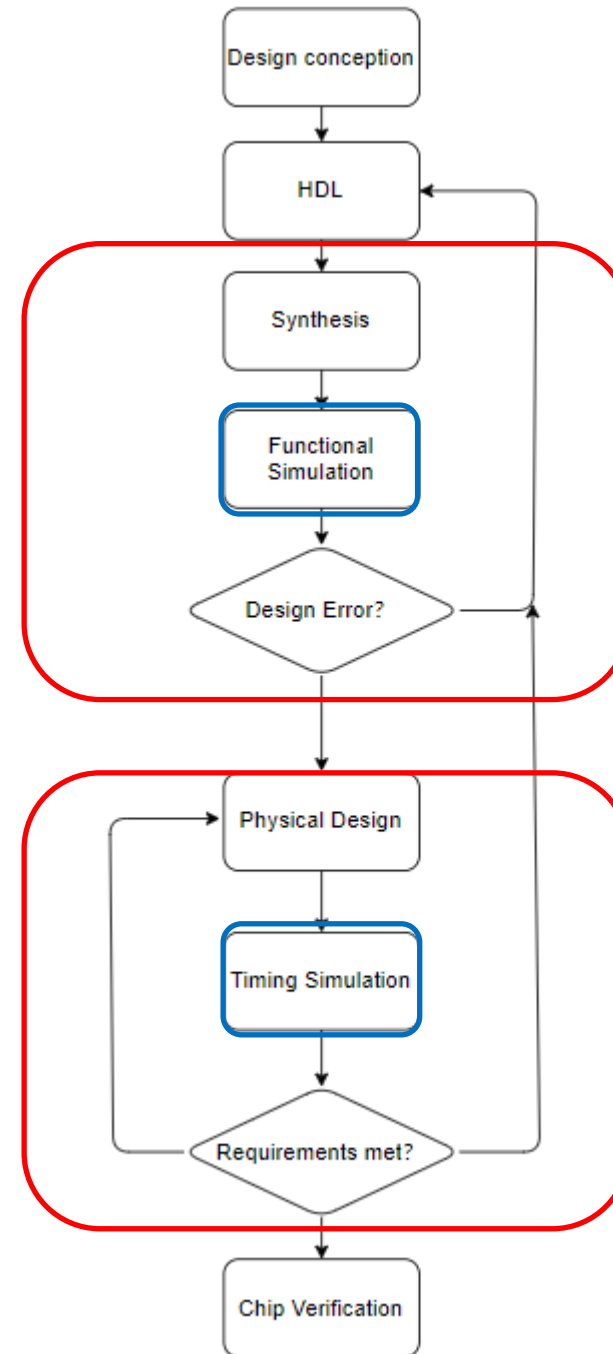
```
library ieee;  
  
use ieee.std_logic_1164.all;  
  
entity led_tb is  
end led_tb;  
  
architecture tb of led_tb is  
  component led  
    port (  
      input : in std_logic;  
      output : out std_logic  
    );  
  end component;  
  
  signal i,o,clk: std_logic;  
  constant time_unit: time := 10 ns;  
  
begin  
  DUT : led port  
    map (  
      input => i,  
      output => o  
    );  
  
  i <= '0', '1' after time_unit, '0' after time_unit*2, '1' after time_unit*3;  
  
end tb;
```

# Review - EDA Flowchart

RTL Analysis  
Synthesis

Implementation

Program and Debug



# VHDL Basic

- 库和程序包的使用
- LIBRARY IEEE
- USE IEEE.Std\_Logic\_1164.all
  - 定义了标准逻辑类型std\_logic / std\_logic\_vector以及相关的类型转换函数
- USE IEEE.Numeric\_Std.all
  - 定义了unsigned和signed两种数据类型，用于表示无符号数和有符号数，并提供了进行算术、逻辑和比较运算的函数和运算符
  - USE IEEE.Std\_Logic\_Arith.all
  - USE IEEE.Std\_Logic\_unsigned.all
  - USE IEEE.Std\_Logic\_signed.all

# VHDL Basic

- 并行语句

- 基本赋值语句



```
a <= '1' after 10 ns, '0' after 20 ns;
```

- 条件赋值语句



```
b <= "11" when sel = "00" else  
    "10" when sel = "01" else  
    "01" when sel = "10" else  
    "00" when sel = "11" else  
    "ZZ";
```

- 选择赋值语句



- 进程语句

- 元件例化语句

```
with sel select  
    c <= "11" when "00",  
        "10" when "01",  
        "01" when "10",  
        "00" when "11",  
        "ZZ" when others;
```

# VHDL Basic

- 进程语句
  - 进程语句定义了一组不断的顺序执行的语句

```
P1: process(clk, rst)  敏感参数列表
-- 变量定义
begin
    -- 顺序语句
    -- 顺序语句
end process;
```

- 顺序语句
  - If语句
  - Case语句

```
process(sel)
begin
    if sel = "00" then
        b <= "11";
    elsif sel = "01" then
        b <= "10";
    elsif sel = "10" then
        b <= "01";
    elsif sel = "11" then
        b <= "00";
    else
        b <= "ZZ";
    end if;
end process;
```

```
process(sel)
begin
    case sel is
        when "00" =>
            c <= "11";
        when "01" =>
            c <= "10";
        when "10" =>
            c <= "01";
        when "11" =>
            c <= "00";
        when others =>
            c <= "ZZ";
    end case;
end process;
```

# Modelling Styles

- Behavioral行为设计
- 关注电路的功能和算法，强调对电路行为的描述

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity FullAdder is
    Port ( a : in std_logic;
           b : in std_logic;
           cin : in std_logic;
           s : out std_logic;
           cout : out std_logic);
end FullAdder;
```

```
architecture Behavioral of FullAdder is
    signal temp_sum : unsigned(1 downto 0);
    signal a_bus: std_logic_vector(1 downto 0);
    signal b_bus: std_logic_vector(1 downto 0);
    signal cin_bus: std_logic_vector(1 downto 0);
begin
    a_bus <= '0' & a;           '0' & a: 在logic信号a的前面拓展一位“0”
    b_bus <= '0' & b;
    cin_bus <= '0' & cin;
    temp_sum <= unsigned(a_bus) + unsigned(b_bus) + unsigned(cin_bus);
    s <= std_logic(temp_sum(0));
    cout <= std_logic(temp_sum(1));
end Behavioral;
```



# Modelling Styles

- Behavioral行为设计
- 关注电路的功能和算法，强调对电路行为的描述

```
architecture Behavioral of FullAdder is
begin
    process(a, b, cin)
    begin
        if (a = '0' and b = '0' and cin = '0') then
            s <= '0';
            cout <= '0';
        elsif (a = '0' and b = '0' and cin = '1') then
            s <= '1';
            cout <= '0';
        elsif (a = '0' and b = '1' and cin = '0') then
            s <= '1';
            cout <= '0';
        elsif (a = '0' and b = '1' and cin = '1') then
            s <= '0';
            cout <= '1';
```

```
        elsif (a = '1' and b = '0' and cin = '0') then
            s <= '1';
            cout <= '0';
        elsif (a = '1' and b = '0' and cin = '1') then
            s <= '0';
            cout <= '1';
        elsif (a = '1' and b = '1' and cin = '0') then
            s <= '0';
            cout <= '1';
        elsif (a = '1' and b = '1' and cin = '1') then
            s <= '1';
            cout <= '1';
        else
            s <= 'X';
            cout <= 'X';
        end if;
    end process;
end Behavioral;
```

# Modelling Styles

- Dataflow数据流设计
- 关注数据的流动和处理，强调输入输出信号之间的逻辑关系。

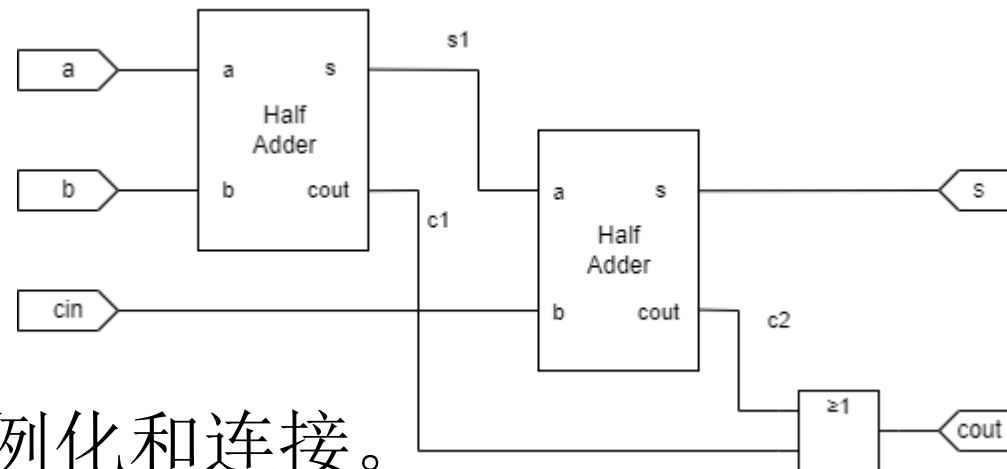
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FullAdder is
    Port ( a : in std_logic;
           b : in std_logic;
           cin : in std_logic;
           s : out std_logic;
           cout : out std_logic);
end FullAdder;

architecture Dataflow of FullAdder is
begin
    s <= a xor b xor cin;
    cout <= (a and b) or (a and cin) or (b and cin);
end Dataflow;
```

# Modelling Styles

- Structural结构设计
- 关注电路的物理结构，强调组件的实例化和连接。



```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity FullAdder is  
    Port ( a : in std_logic;  
           b : in std_logic;  
           cin : in std_logic;  
           s : out std_logic;  
           cout : out std_logic);  
end FullAdder;
```

```
architecture Structural of FullAdder is  
    component HalfAdder is  
        Port ( a : in std_logic;  
              b : in std_logic;  
              s : out std_logic;  
              cout : out std_logic);  
    end component;  
    signal s1, c1, c2 : std_logic;
```

```
begin  
    HA1: HalfAdder  
        Port map (  
            a => a,  
            b => b,  
            s => s1,  
            cout => c1  
        );  
    HA2: HalfAdder  
        Port map (  
            a => s1,  
            b => cin,  
            s => s,  
            cout => c2  
        );  
    cout <= c1 or c2;  
end Structural;
```

<下层实体端口名> => <本层信号名>

结构设计：通常用于顶层设计，进行功能模块之间的连接

数据流设计：通常用于简单的模块的设计

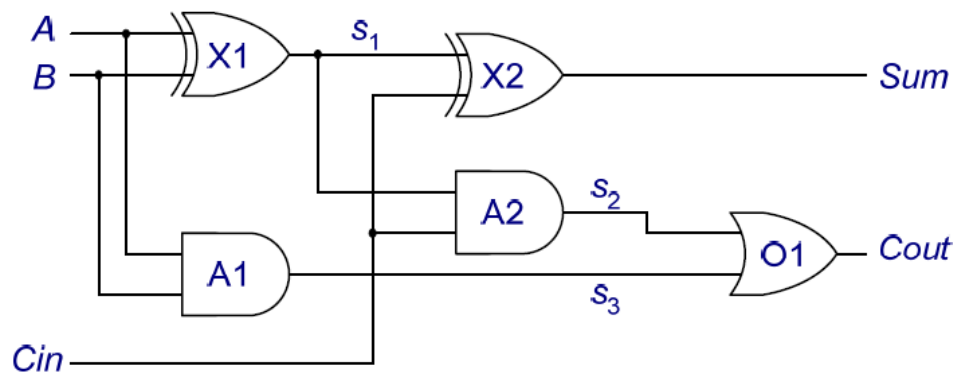
行为设计：通常用于功能较为复杂的模块的设计

# Assignment 1

请在Vivado中对全加器进行设计仿真，全加器采用代码如下：

```
library IEEE;
use IEEE.std_logic_1164.all;

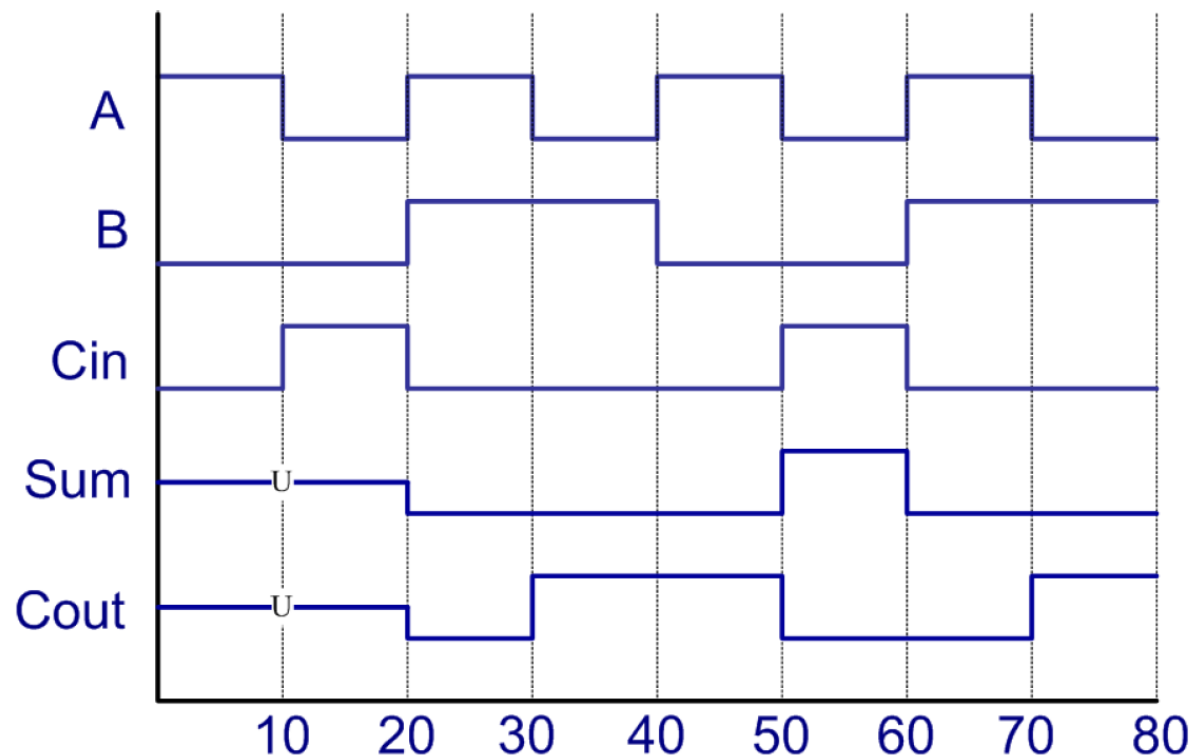
entity full_adder is
port (A, B, Cin : in std_logic;
      Sum, Cout: out std_logic);
end entity full_adder;
```



```
architecture dataflow of full_adder is
signal s1, s2, s3 : std_logic;
constant gate_delay : time := 10 ns;
begin
    L1: s1 <= (A xor B) after gate_delay;
    L2: s2 <= (Cin and s1) after gate_delay;
    L3: s3 <= (A and B) after gate_delay;
    L4: sum <= (s1 xor Cin) after gate_delay;
    L5: cout <= (s2 or s3) after gate_delay;
end architecture dataflow;
```

# Assignment 1

请编写testbench文件，用下图中的A、B、Cin来进行激励。请分别进行behavior simulation、post synthesis simulation、以及post implementation simulation，请根据以下问题分析这些结果的特点和异同。



# Assignment 1

1. 仿真结果是否与上页图中的结果一致？
2. 加法计算结果是否和人们通常的认知一致？
3. 如不一致，导致不一致的原因是什么？
4. 3种仿真结果有什么不同？
5. 为什么会有不同？
6. 你是否在结果看了一些尖峰？
7. 这些尖峰是由什么引起的？

本题目请提交完整报告，包括：题目、实验目的、VHDL代码和testbench代码（合理注释）、结果（schematic、仿真波形）、分析、结论

# Assignment 2

请通过以下程序理解变量和信号的区别

```
library IEEE;
use IEEE.std_logic_1164.all;

entity sig_var is
port (x, y, z : in std_logic;
      res1, res2 : out std_logic);
end entity sig_var;
```

```
architecture dataflow of sig_var is
signal sig_s1, sig_s2 : std_logic;
begin
  proc1: process (x, y, z) is
  variable var_s1, var_s2 : std_logic;
  begin
    L1: var_s1 := x and y;
    L2: var_s2 := var_s1 xor z;
    L3: res1 <= var_s1 nand var_s2;
  end process proc1;

  proc2: process (x, y, z) is
  begin
    L1: sig_s1 <= x and y;
    L2: sig_s2 <= sig_s1 xor z;
    L3: res2 <= sig_s1 nand sig_s2;
  end process proc2;
end architecture dataflow;
```

# Assignment 2

1. 假设x、y、z信号为初始值为0、占空比为50%的周期方波，其中x信号的周期为 80 ns，y信号的周期为40 ns，z信号的周期为 20ns。
2. 请自行推导res1和res2的输出波形。
3. 请设计testbench文件进行仿真以验证你的结果。

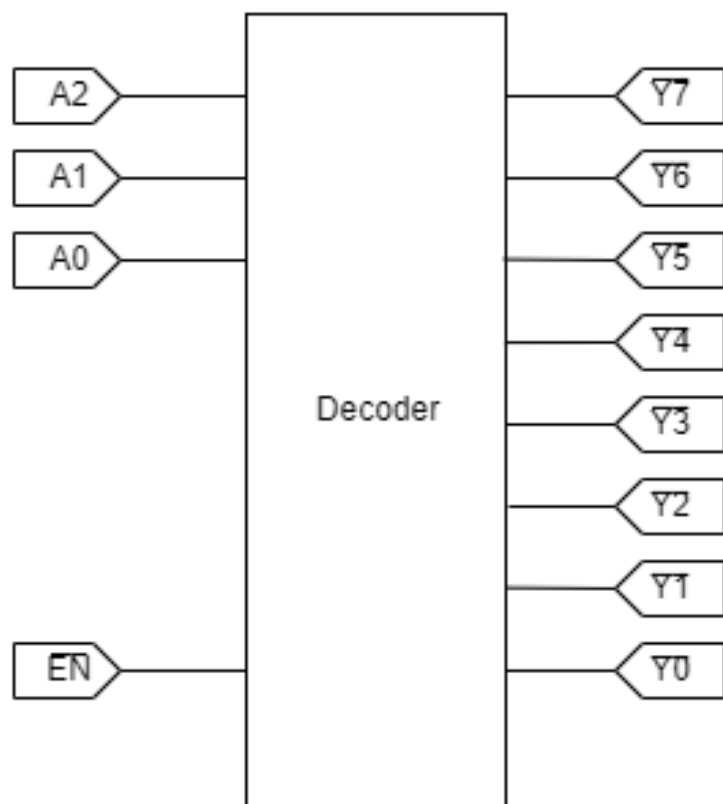
本题目请提交简易报告，包括：题目、 testbench代码（合理注释）、结果

(截图)



# Assignment 3

请设计一个3-8译码器，其功能表如下。



| A  |    |    | EN | Y  |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|
| A2 | A1 | A0 |    | Y7 | Y6 | Y5 | Y4 | Y3 | Y2 | Y1 | Y0 |
| X  | X  | X  | 1  | 1  | 1  | 1  | 1  | 1  | 1  | 1  | 1  |
| 0  | 0  | 0  | 0  | 1  | 1  | 1  | 1  | 1  | 1  | 1  | 0  |
| 0  | 0  | 1  | 0  | 1  | 1  | 1  | 1  | 1  | 1  | 0  | 1  |
| 0  | 1  | 0  | 0  | 1  | 1  | 1  | 1  | 1  | 0  | 1  | 1  |
| 0  | 1  | 1  | 0  | 1  | 1  | 1  | 1  | 0  | 1  | 1  | 1  |
| 1  | 0  | 0  | 0  | 1  | 1  | 1  | 0  | 1  | 1  | 1  | 1  |
| 1  | 0  | 1  | 0  | 1  | 1  | 0  | 1  | 1  | 1  | 1  | 1  |
| 1  | 1  | 0  | 0  | 1  | 0  | 1  | 1  | 1  | 1  | 1  | 1  |
| 1  | 1  | 1  | 0  | 0  | 1  | 1  | 1  | 1  | 1  | 1  | 1  |

# Assignment 3

本题目需进行验收。

使用SW2~SW0对应A2~A0， SW3对应EN， LED7~LED0对应Y7~Y0。

请说明实现方法，展示RTL原理图，演示其功能。